Universidade Estadual de Campinas (Unicamp)

IMECC – Instituto de Matemática, Estatística e Computação Científica

Introdução ao Aprendizado de Máquinas Profundo – MS671

Reconhecimento de Objetos Usando YOLO

Nome:
Jhonatan Bryan Lemmo Rodrigues

RA: 199543

Sumário

1 Introdução **2**
1.1 Importância e Motivação 2
1.2 Evolução dos Métodos 2

2 Arquitetura e Algoritmo YOLO **3**
2.1 Visão Geral do Pipeline 3
2.2 Pré-processamento: Letterbox 3
2.3 Backbone: Darknet-53 3
2.4 Cabeçalho de Detecção em Múltiplas Escalas 4
2.5 Âncoras 4
2.6 Supressão Não-Maximal (NMS) 4
2.6.1 Cálculo do IoU 5

3 Implementação **5**
3.1 Estrutura do Código 5

4 Testes **5**

5 Resultados **7**
5.1 Imagens dos Resultados 8

6 Teste do Algoritmo com Imagens Reais **9**

7 Discussão **11**
7.1 Análise dos Resultados 11
7.2 Possíveis Melhorias 12

8 Conclusão **12**

1 Introdução

A detecção de objetos é uma das tarefas mais importantes da visão computacional, com aplicações que vão desde sistemas de vigilância e veículos autônomos até diagnósticos médicos e robótica. Diferente da classificação de imagens, que simplesmente atribui um rótulo à imagem inteira, a detecção precisa ao mesmo tempo localizar cada objeto de interesse e classificá-lo, devolvendo como saída um conjunto de caixas delimitadoras com suas respectivas probabilidades de classe.

1.1 Importância e Motivação

O avanço dos métodos modernos de detecção abriu caminho para diversas aplicações:

- **Veículos autônomos:** detecção de pedestres, ciclistas e sinais de trânsito em tempo real exige processamento de múltiplos quadros por segundo com alta precisão.
- **Segurança e vigilância:** câmeras inteligentes identificam comportamentos suspeitos ou contam pessoas em multidões.
- **Saúde:** detecção automatizada de nódulos pulmonares, lesões de pele e anomalias em exames de imagem aceleram o diagnóstico clínico.
- **Varejo e logística:** contagem de itens em prateleiras, rastreamento de pacotes e automação de armazéns.
- **Agricultura de precisão:** identificação de pragas, doenças e estimativa de colheitas por drones.

1.2 Evolução dos Métodos

Os métodos de detecção evoluíram basicamente em duas grandes gerações:

Métodos de duas etapas (*two-stage detectors*). Abordagens como R-CNN [3] primeiro propõem regiões candidatas e depois as classificam. Embora muito precisos, são geralmente lentos demais para aplicações em tempo real.

Métodos de etapa única (*one-stage detectors*). A YOLO (*You Only Look Once*) [1] reformula a detecção como um problema direto de regressão sobre uma grade, eliminando a etapa de proposta de região. O resultado é uma velocidade muito superior, com perda relativamente pequena de precisão.

O algoritmo YOLO, introduzido por Redmon et al. em 2016, é hoje um dos detectores de etapa única mais usados no mundo. A terceira versão, YOLOv3 [2], trouxe conexões residuais no estilo ResNet, predição em múltiplas escalas e uso de sigmoide para classificação multilabel, o que o torna bem mais robusto para objetos de tamanhos variados.

2 Arquitetura e Algoritmo YOLO

2.1 Visão Geral do Pipeline

O YOLOv3 processa uma imagem de entrada de tamanho fixo (tipicamente 416 × 416 pixels) e produz previsões em três escalas de resolução, capturando objetos grandes, médios e pequenos. O pipeline completo consiste em:

1. Pré-processamento da imagem com *letterboxing*.
2. Extração de características pela *backbone* Darknet-53.
3. Detecção em três escalas via cabeçotes YOLO.
4. Decodificação das previsões (centro, dimensões, confiança, classes).
5. Filtragem por limiar de confiança.
6. Supressão não-maximal (*Non-Maximum Suppression* – NMS).
7. Visualização dos resultados.

2.2 Pré-processamento: Letterbox

Em vez de redimensionar a imagem diretamente (o que deformaria a proporção), aplica-se o **letterbox**: escala-se a imagem de forma que o lado maior caiba em 416 pixels, e o espaço restante é preenchido com barras cinzas (valor 128), mantendo a proporção original. Após a inferência, um passo inverso remove a influência dessas barras das coordenadas previstas.

Formalmente, seja (W_0, H_0) o tamanho original e $S = 416$. A escala aplicada é

$$s = \min\left(\frac{S}{W_0}, \frac{S}{H_0}\right), \tag{1}$$

e as novas dimensões são $(n_w, n_h) = (\lfloor s\, W_0 \rfloor, \lfloor s\, H_0 \rfloor)$.

2.3 Backbone: Darknet-53

A *backbone* é responsável por extrair representações hierárquicas da imagem. A Darknet-53 é uma rede convolucional profunda com 53 camadas convolucionais, organizadas em blocos residuais inspirados no ResNet [5]:

- **ConvBlock:** Convolução → Batch Normalization → LeakyReLU($\alpha = 0{,}1$).
- **ResBlock:** ConvBlock(1×1) → ConvBlock(3×3) + conexão de atalho.

A Tabela 1 resume as camadas da Darknet-53:

Tabela 1: Estrutura resumida da Darknet-53 usada como *backbone* do YOLOv3.

Camada	**Filtros**	**Tamanho**	**Stride**	**Saída (entrada 416)**	**Repetições**
ConvBlock	32	3×3	1	416×416	1
Residual-1	64	3×3	2	208×208	1
Residual-2	128	3×3	2	104×104	2
Residual-3	256	3×3	2	52×52	8
Residual-4	512	3×3	2	26×26	8
Residual-5	1024	3×3	2	13×13	4

As saídas de Residual-3 (52×52), Residual-4 (26×26) e Residual-5 (13×13) são usadas como rotas para as três escalas de detecção – pequena, média e grande, respectivamente.

2.4 Cabeçalho de Detecção em Múltiplas Escalas

Cada escala possui:

1. Um bloco de 5 convoluções intercaladas (1×1 e 3×3), denominado C5, que refina as características.

2. Uma convolução linear final (sem Batch Normalization) que produz o tensor de previsão com dimensão $3 \times (5 + C)$ por posição de grade, onde $C = 80$ é o número de classes COCO.

2.5 Âncoras

Cada posição de grade prediz 3 caixas, uma para cada âncora pré-definida. As 9 âncoras do YOLOv3 (obtidas via *k-means* no dataset COCO [4]) são apresentadas na Tabela 2.

Tabela 2: Âncoras do YOLOv3 (largura $\times$ altura, em pixels para entrada 416).

Escala	**Grade**	**Âncoras (px)**
1 – Objetos Grandes	13×13	(116, 90), (156, 198), (373, 326)
2 – Objetos Médios	26×26	(30, 61), (62, 45), (59, 119)
3 – Objetos Pequenos	52×52	(10, 13), (16, 30), (33, 23)

2.6 Supressão Não-Maximal (NMS)

Após filtrar detecções pelo limiar de confiança θ_s, múltiplas caixas podem corresponder ao mesmo objeto. O algoritmo de NMS procede da seguinte forma:

1. Ordena as caixas por escore decrescente.

2. Seleciona a caixa de maior escore e a mantém.

3. Remove todas as caixas cuja Interseção sobre União (IoU) com a caixa selecionada supere o limiar θ_{IoU}.

4. Repete a partir do passo 2 com as caixas restantes.

O NMS é executado por classe, o que evita suprimir detecções legítimas de classes diferentes que se sobreponham espacialmente.

2.6.1 Cálculo do IoU

Dadas duas caixas A e B representadas por $[y_{\min}, x_{\min}, y_{\max}, x_{\max}]$:

$$\mathrm{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|} = \frac{|A \cap B|}{|A| + |B| - |A \cap B|}. \tag{2}$$

A interseção é calculada como:

$$x_{\text{inter}} = \max\Big(0,\ \min(x^A_{\max},\ x^B_{\max}) - \max(x^A_{\min},\ x^B_{\min})\Big)\,, \tag{3}$$

$$y_{\text{inter}} = \max\Big(0,\ \min(y^A_{\max},\ y^B_{\max}) - \max(y^A_{\min},\ y^B_{\min})\Big)\,, \tag{4}$$

$$|A \cap B| = x_{\text{inter}} \times y_{\text{inter}}. \tag{5}$$

3 Implementação

3.1 Estrutura do Código

O projeto é organizado nos seguintes módulos principais:

`YOLOv3` Classe principal da rede, implementada em PyTorch, contendo a Darknet-53 e os três cabeçotes de detecção.

`carregar_pesos_yolov3` Função que lê o arquivo binário `.weights` e injeta os parâmetros na estrutura PyTorch, respeitando a ordem: bias BN → peso BN → média BN → variância BN → pesos Conv.

`letterbox_image` / `reverter_escala_caixas` Funções de pré e pós-processamento que preservam a proporção da imagem.

`decode_yolo` Decodifica os tensores brutos de saída de cada escala para coordenadas e *scores* absolutos.

`manual_nms` / `compute_iou` Implementação manual e vetorizada do NMS por classe.

`executar_predicao` Orquestra todo o pipeline de inferência e visualização dos resultados.

4 Testes

Os experimentos foram conduzidos com os pesos pré-treinados `yolov3.weights` (treinados no dataset COCO com 80 classes) no ambiente Google Colab. Queremos testar o algoritmo para diferentes limiares de probabilidade de confiança (0; 0,25; 0,5; 0,75; 0,95) e de IoU (0,3; 0,7; 1,0). Para isso usamos duas imagens como base:

- A Figura 1 é uma mesa cheia de pratos, tigelas e comidas.

- A Figura 2 é uma imagem de trânsito retirada da internet, com vários carros e motos.

Essas imagens foram escolhidas justamente por serem complicadas para o algoritmo, testando seus limites. Limitamos a 50 caixas detectadas.



Figura 1: Imagem de uma mesa repleta de recipientes e alimentos a serem identificados.



Figura 2: Imagem de trânsito retirada da internet com múltiplos veículos e pedestres a serem identificados.

Do conjunto de 80 classes treinadas disponíveis, vamos usar como base que temos 6 colheres, 4 tigelas e uma mesa de jantar para a Figura 1, e 5 pessoas, 5 carros, 2 motos, 2 caminhões e 2 semáforos para a Figura 2. Até mesmo para um humano há dificuldade

em classificar os objetos nessas imagens: até onde consideramos tigelas diferentes de pratos, ou conchas de colheres na Figura 1; e caso observarmos com atenção e zoom na Figura 2, vemos mais carros e semáforos ao fundo que são difíceis de perceber.

Por fim comparamos com o modelo mais recente (Yolo26n) com o código 'cru' disponibilizado no site da Ultralytics [6].

5 Resultados

As Tabelas 3 e 4 apresentam as contagens de objetos detectados para a imagem da mesa e a de trânsito, variando os limiares de confiança e IoU.

Tabela 3: Detecções obtidas da Figura 1 para diferentes limiares de IoU e confiança.

IoU	**Confiança = 0**	**Confiança = 0,25**	**Confiança = 0,5**	**Confiança = 0,75**	**Confiança = 0,95**
0,3	garfo: 1 colher: 11 tigela: 18 brócolis: 19 mesa_jantar: 1	garfo: 1 colher: 7 tigela: 15 brócolis: 1 mesa_jantar: 1	garfo: 1 colher: 6 tigela: 10 mesa_jantar: 1	colher: 2 tigela: 5 mesa_jantar: 1	tigela: 4
0,7	garfo: 1 colher: 23 tigela: 23 brócolis: 2 mesa_jantar: 1	garfo: 1 colher: 18 tigela: 21 brócolis: 1 mesa_jantar: 1	garfo: 1 colher: 9 tigela: 12 mesa_jantar: 1	colher: 3 tigela: 6 mesa_jantar: 1	tigela: 4
1,0	garfo: 1 colher: 18 tigela: 28 brócolis: 1 mesa_jantar: 2	garfo: 1 colher: 18 tigela: 28 brócolis: 1 mesa_jantar: 2	garfo: 1 colher: 12 tigela: 21 mesa_jantar: 2	colher: 4 tigela: 12 mesa_jantar: 1	tigela: 7

Para o Yolo26n os resultados foram 4 colheres, 10 tigelas, 3 cenouras e 2 mesas de jantar.

Tabela 4: Detecções obtidas da Figura 2 para diferentes limiares de IoU e confiança.

IoU	**Confiança = 0**	**Confiança = 0,25**	**Confiança = 0,5**	**Confiança = 0,75**	**Confiança = 0,95**
0,3	pessoa: 20 carro: 19 moto: 3 caminhão: 3 semáforo: 5	pessoa: 7 carro: 7 moto: 2 caminhão: 2 semáforo: 2	pessoa: 5 carro: 6 moto: 2 caminhão: 1 semáforo: 1	pessoa: 3 carro: 6 moto: 2 caminhão: 1	pessoa: 2 carro: 4 moto: 1 caminhão: 1
0,7	pessoa: 17 carro: 17 moto: 5 caminhão: 6 semáforo: 5	pessoa: 12 carro: 13 moto: 5 caminhão: 4 semáforo: 3	pessoa: 6 carro: 7 moto: 5 caminhão: 1 semáforo: 1	pessoa: 3 carro: 6 moto: 4 caminhão: 1	pessoa: 2 carro: 4 moto: 1 caminhão: 1
1,0	pessoa: 14 carro: 19 moto: 10 caminhão: 6 semáforo: 1	pessoa: 14 carro: 19 moto: 10 caminhão: 6 semáforo: 1	pessoa: 12 carro: 17 moto: 10 caminhão: 6 semáforo: 1	pessoa: 9 carro: 14 moto: 8 caminhão: 4	pessoa: 5 carro: 6 moto: 2 caminhão: 2

Para o Yolo26n os resultados foram 5 pessoas, 7 carros, 4 motos, 2 caminhões e 1 semáforo.

5.1 Imagens dos Resultados

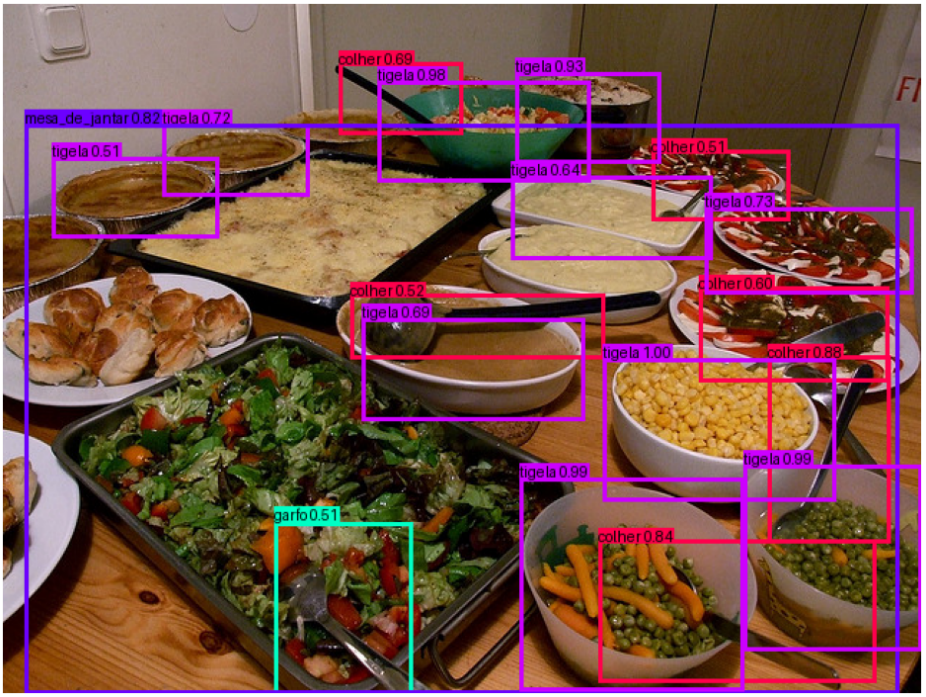


Figura 3: Resultado do YOLOv3 para a imagem da mesa com IoU = 0,3 e confiança = 0,5.

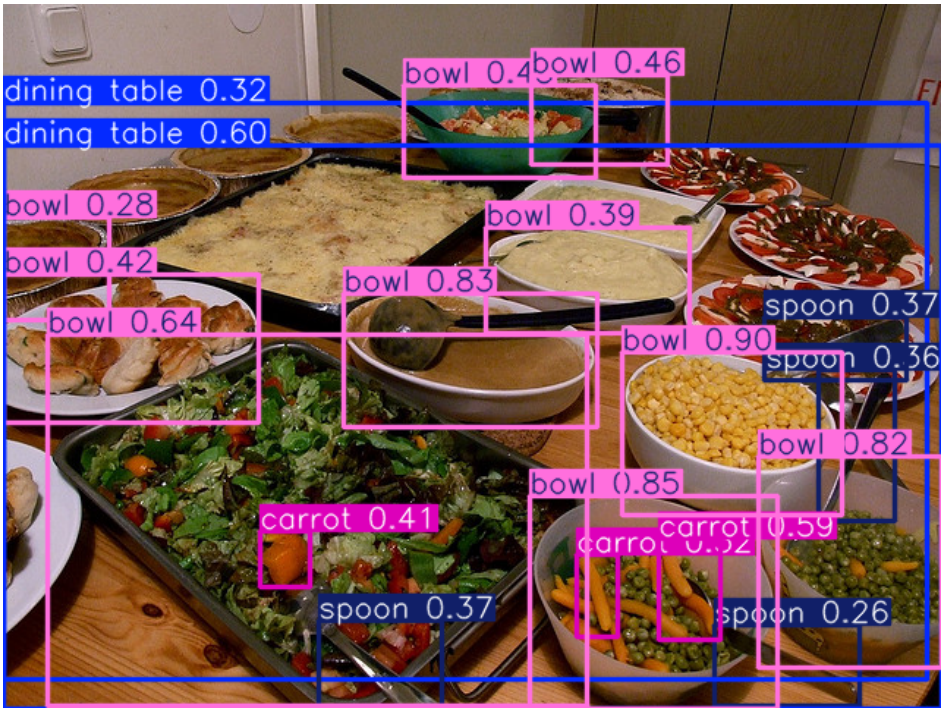


Figura 4: Resultado para a imagem da mesa utilizando o Yolo26n.

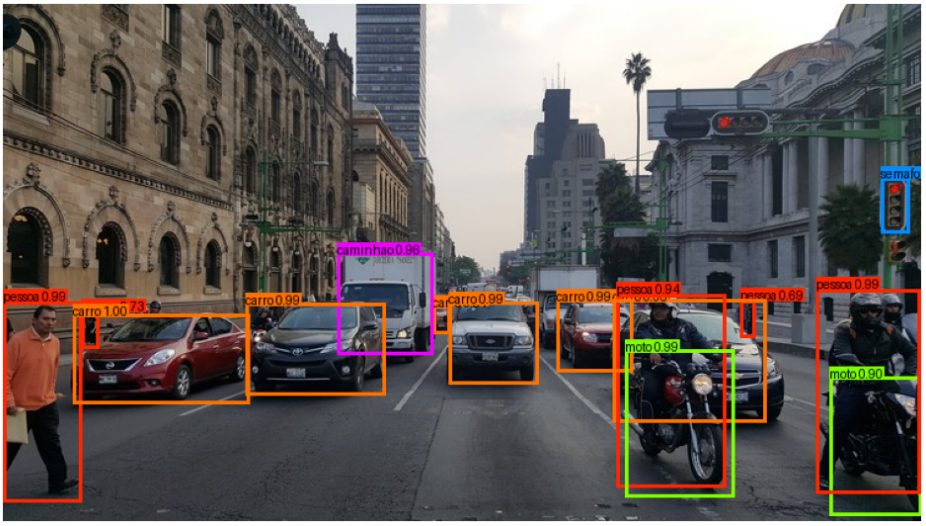


Figura 5: Resultado do YOLOv3 para a imagem de trânsito com IoU = 0,3 e confiança = 0,5.

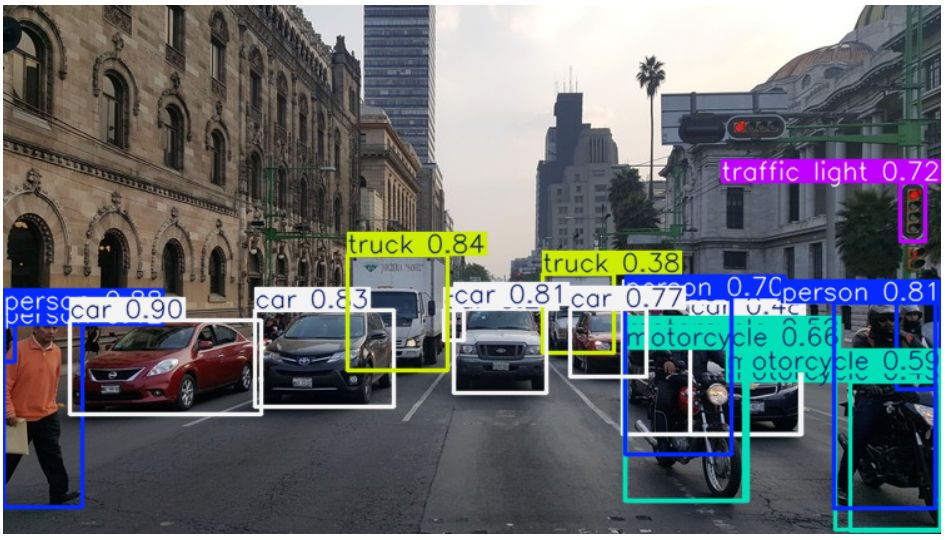


Figura 6: Resultado para a imagem de trânsito utilizando o Yolo26n.

6 Teste do Algoritmo com Imagens Reais

Foram retiradas fotos reais usando um celular Samsung A55 para classificá-las, todas usando IoU = 0,3 e confiança = 0,5.

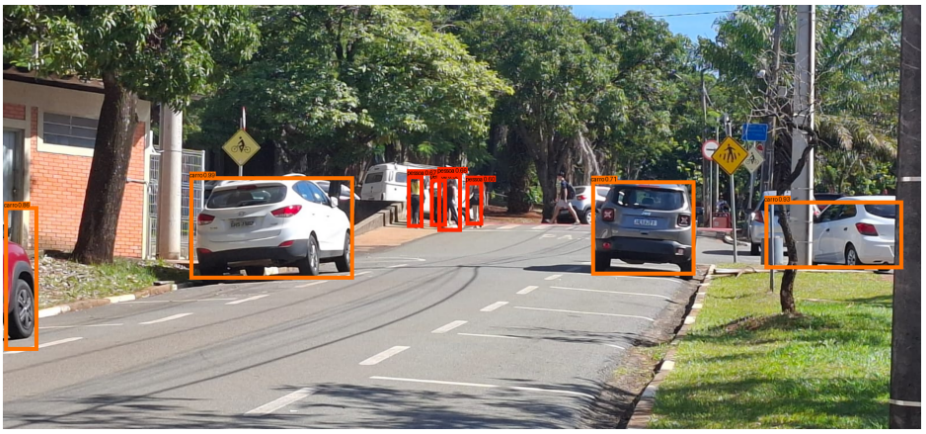


Figura 7: Detecção em imagem real de rua com carros estacionados.



Figura 8: Detecção em imagem real de via pública com veículos.



Figura 9: Detecção em imagem real de área aberta com poucos objetos.

Em geral o algoritmo detectou corretamente os objetos da imagem, alguns com objetos na frente não foi detectado claramente com uma boa precisão

7 Discussão

7.1 Análise dos Resultados

Em ambas as imagens os melhores resultados ficaram com IoU = 0,3 e confiança = 0,5, comparando com nossa base de classificação no olho. Para confiança ganhamos certeza no que realmente existe, porém alguns que estão certos deixam de ser detectados. Já o IoU não tem ganho ao aumentar o limiar, visto que só gera mais caixas sobrepostas, como mostra a Tabela 5.

Tabela 5: Efeito do limiar de IoU sobre a qualidade das detecções.

IoU	Observação
0,3	Resultado mais coerente com a referência visual
0,7	Início de caixas duplicadas visíveis
1,0	Muitas caixas sobrepostas para o mesmo objeto

Para valores intermediários de confiança e IoU em imagens mais complexas, o algoritmo teve boa performance mesmo com alguns erros, pois foram usadas imagens que até um humano tem dificuldade em classificar. Na Figura 3, o reflexo da luz na salada reflete na colher e o algoritmo entende como um garfo; em graus de confiança menores, o verde da salada é reconhecido como brócolis.

Em imagens bem iluminadas e com poucos objetos, a detecção é perfeita, como na Figura 10.

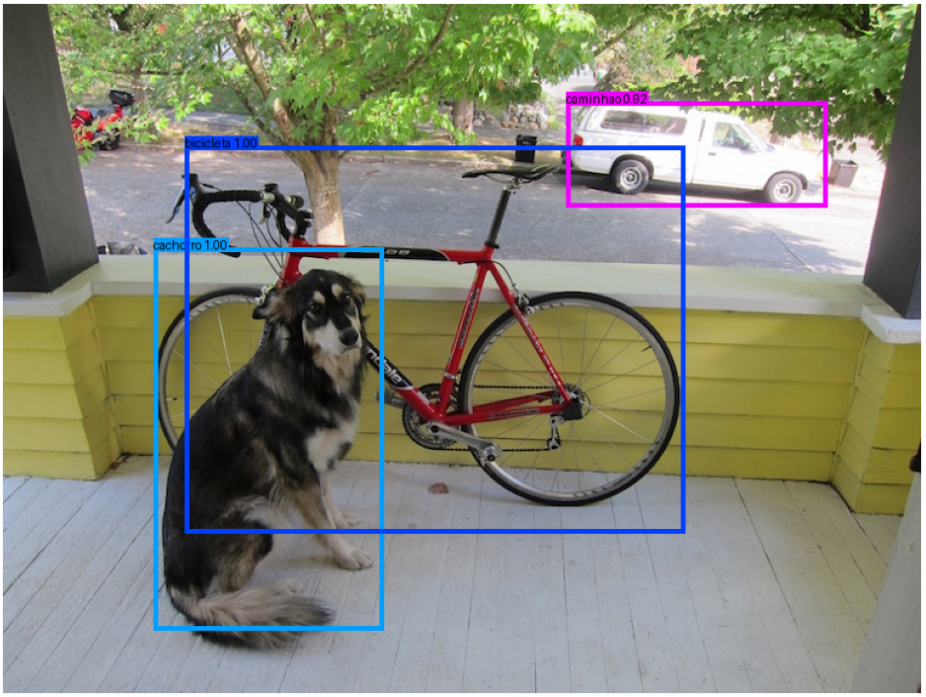


Figura 10: Imagem com poucos objetos bem iluminados, classificados corretamente pelo YOLOv3 alta confiança.

A versão do Yolo26n 'cru' tem maior quantidade de classes, porém é visível que o IoU não está correto, há caixas se sobrepondo onde não deveriam (vide Figuras 4 e 6).

7.2 Possíveis Melhorias

- **Aumento de dados:** aumentar a diversidade de escala dos exemplos de treinamento.
- **Aumento no número de classes:** treinar com mais classes de objetos, já disponível em versões mais recentes do YOLO.
- **Ajuste fino:** para domínios específicos (ex: células médicas, peças industriais), o ajuste fino dos pesos COCO em datasets especializados melhora significativamente o desempenho.

8 Conclusão

Este projeto implementou com sucesso o pipeline completo de detecção de objetos com YOLOv3 em PyTorch, incluindo a arquitetura Darknet-53, o carregamento dos pesos pré-treinados a partir do formato binário original, o pré e pós-processamento com *letterboxing*, e uma implementação manual e vetorizada do NMS por classe.

Os resultados obtidos nas imagens de teste mostram que o YOLOv3 funciona bem como detector em tempo real, com detecções precisas para objetos de tamanho mediano e grande. A análise dos limiares de confiança e IoU mostrou a importância de calibrá-los de acordo com a aplicação: onde falsos positivos têm alto custo (ex: diagnósticos) convém usar confiança elevada, enquanto onde falsos negativos são mais prejudiciais (ex: segurança) convém usar confiança mais baixa.

Os testes com imagens reais confirmaram que o algoritmo generaliza bem para cenas do cotidiano. A comparação com o Yolo26n mostrou que versões mais recentes têm mais classes disponíveis, mas que calibrar corretamente os parâmetros de NMS continua sendo essencial independente da versão.

Referências

[1] Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). *You Only Look Once: Unified, Real-Time Object Detection*. Proceedings of CVPR 2016.

[2] Redmon, J., & Farhadi, A. (2018). *YOLOv3: An Incremental Improvement*. arXiv preprint arXiv:1804.02767.

[3] Girshick, R., Donahue, J., Darrell, T., & Malik, J. (2014). *Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation*. Proceedings of CVPR 2014.

[4] Lin, T.-Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P., & Zitnick, C. L. (2014). *Microsoft COCO: Common Objects in Context*. Proceedings of ECCV 2014.

[5] He, K., Zhang, X., Ren, S., & Sun, J. (2016). *Deep Residual Learning for Image Recognition*. Proceedings of CVPR 2016.

[6] Ultralytics. *Model Prediction with Ultralytics YOLO*. Disponível em: https://docs.ultralytics.com/modes/predict/#key-features-of-predict-mode. Acesso em: mai. 2025.